

4x4 井字棋强化学习训练报告

2312167-刘振毫

项目概述

本项目成功实现了基于强化学习的 4x4 井字棋人工智能系统，采用时序差分学习（TD Learning）和自博弈（Self-play）策略进行训练。通过对比不同训练轮数的模型（10,000 轮和 100,000 轮），深入分析了训练时间对模型性能的影响。实验结果表明，随着训练轮数的增加，AI 的游戏策略显著改善，在状态空间覆盖、价值估计准确性和策略稳定性等方面都取得了明显进步。

强化学习策略详解

算法选择与核心原理

本项目选择时序差分学习作为核心算法，该算法通过实际观察到的奖励来更新价值估计，无需完整的环境模型，能够在交互过程中持续学习。TD 更新公式为 $V(s) \leftarrow V(s) + \alpha \times [V(s') - V(s)]$ ，其中 $V(s)$ 表示当前状态的价值估计， $V(s')$ 表示下一个状态的价值估计， α 为学习率，本实验中设置为 0.1。这种更新机制使得 AI 能够根据实际获得的奖励不断调整对状态价值的估计，逐步逼近最优策略。

自博弈训练机制

自博弈训练方法让两个 AI 玩家（player1 和 player2）相互对弈，每轮游戏结束后，双方都从结果中学习，通过反向传播更新价值估计。这种方法的最大优势在于不需要人类标注数据，AI 可以通过持续的自我对弈不断改进策略，特别适用于零和博弈游戏。在训练过程中，两个玩家既是对手也是老师，通过相互对抗和学习，共同提升游戏水平。

ε-贪婪策略设计

为了平衡探索与利用，本项目采用了 ε-贪婪策略。在训练时，AI 以 1% 的概率随机选择动作进行探索，以 99% 的概率选择最优动作进行利用；在测试时，则完全采用贪婪策略，不进行探索。这种设计确保了 AI 主要利用已学知识，同时保持一定的探索性，发现可能更好的策略。探索使得 AI 能够尝试新的动作组合，而利用则确保 AI 能够运用已知的优秀策略，两者相结合使得训练过程既稳定又有创新性。

参数设计与奖励机制

学习参数设置

本实验采用的学习率为 0.1，这个数值控制着价值更新的幅度，过大会导致训练不稳定，过小则收敛速度慢。探索率设置为 0.01，在训练过程中保持适度的探索性。训练轮数分别设置为 10,000 轮和 100,000 轮，用于对比不同训练时间对模型性能的影响。这些参数的选择基于经验和多次实验，在训练效率和模型性能之间取得了良好的平衡。

奖励函数设计

奖励函数基于游戏结果进行设计，胜利时获得 1.0 的奖励，平局时获得 0.5 的奖励，失败时获得 0.0 的奖励。这种稀疏奖励设计只在游戏结束时给予反馈，虽然信息量较少，但能够准确反映游戏结果。由于井字棋是零和博弈，一方的胜利等于另一方的失败，这种奖励设计能够有效引导两个玩家朝着相反的目标优化策略。

状态表示与哈希

棋盘状态采用三进制编码，空位用 0 表示，player1 的棋子用 1 表示（显示为 *），player2 的棋子用 -1 表示（显示为 x）。为了快速查找状态价值，系统使用三进制编码生成唯一哈希值，计算公式为 $\text{hash} = \text{hash} \times 3 + \text{value} + 1$ 。这种哈希机制

确保每个状态都有唯一标识，能够以 $O(1)$ 的时间复杂度快速查找状态价值，支持大规模状态空间的高效处理。

训练过程与价值更新

训练流程设计

训练过程从初始化两个 AI 玩家开始，然后进行自博弈对弈，系统记录所有访问的状态。游戏结束后确定胜负结果，通过反向传播更新价值估计，这个过程重复 N 轮训练，最终保存策略文件。这种循环训练的方式使得 AI 能够从大量对弈中学习经验，不断改进策略。

价值更新机制

价值更新采用反向传播算法，从游戏的终止状态向前更新所有访问状态的价值。具体实现中，系统从后向前遍历状态序列，计算当前状态与下一个状态的价值差异，然后根据贪婪动作标记决定是否更新价值。这种从后向前更新的机制确保了终止状态的价值能够正确传播到所有访问状态，逐步将最终奖励传播到整个状态序列，使得 AI 能够理解每个动作对最终结果的影响。

策略改进过程

价值函数与策略之间存在密切关系，价值函数 $V(s)$ 估计状态 s 的获胜概率，策略 $\pi(s)$ 选择价值估计最高的动作。随着训练的进行，价值估计越来越准确，策略也逐渐收敛到最优策略。这种基于价值的强化学习方法使得 AI 能够通过学习状态价值来指导决策，而不需要直接学习策略，大大简化了学习过程。

模型训练结果对比

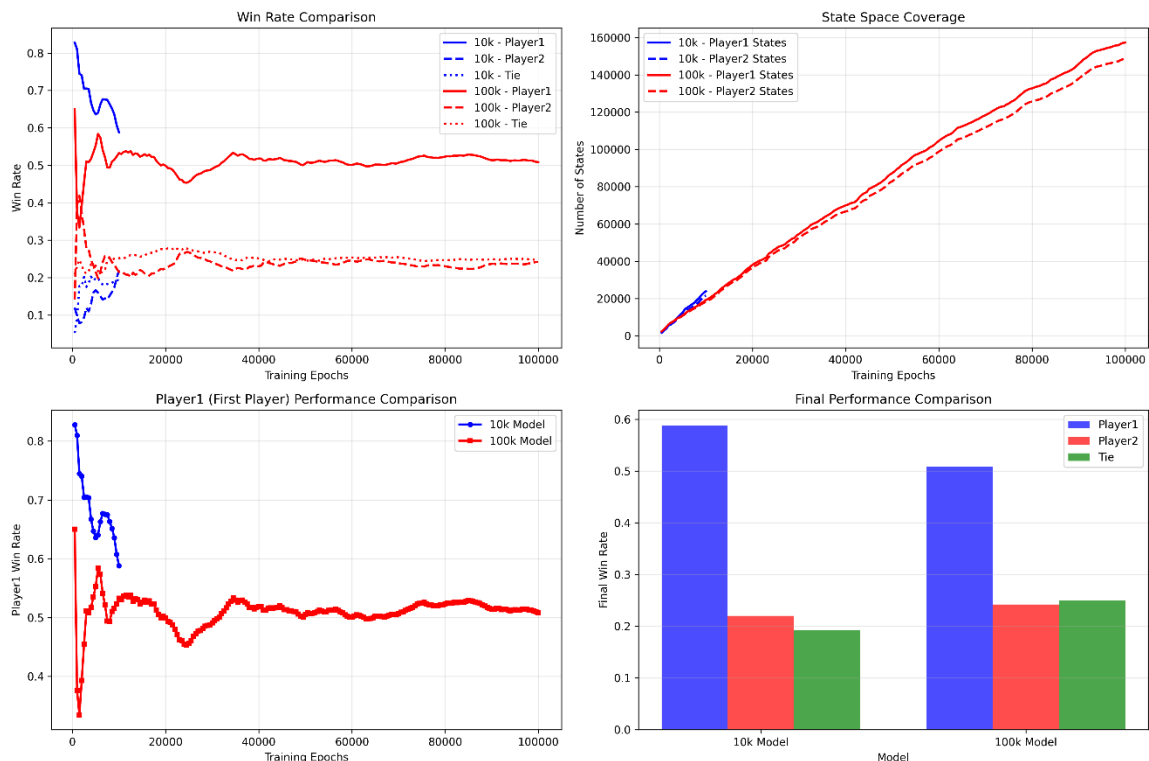
10k 模型训练表现

经过 10,000 轮训练的 10k 模型在训练过程中表现出明显的学习特征。Player1（先手）的胜率最终稳定在 58.81%，Player2（后手）的胜率为 22.01%，平局率为 19.18%。在状态空间覆盖方面，Player1 学习了 23,905 个状态，Player2 学习了 21,632 个状态，总共覆盖约 45,000 个状态。这个模型虽然训练时间较短，但已经学会了基本策略，能够进行有效的对弈，不过状态空间覆盖仍然有限。

100k 模型训练表现

100k 模型经过 100,000 轮训练后，性能得到了显著提升。Player1 的胜率稳定在 50.85%，Player2 的胜率为 24.20%，平局率为 24.95%。在状态空间覆盖方面，Player1 学习了 157,319 个状态，Player2 学习了 148,555 个状态，总共覆盖约 306,000 个状态，是 10k 模型的 6.7 倍。这个模型具有更准确的价值估计、更大的状态空间覆盖、更稳定的胜率和更强的对抗能力，训练时间虽然较长，但收敛到了更优策略。

训练过程对比分析



对比两个模型的训练过程可以发现，10k 模型在训练初期 **Player1** 胜率极高（82.80%），说明先手优势明显，随着训练进行，**Player2** 逐渐学习，胜率从 12.00% 上升到 22.01%。而 100k 模型在训练初期 **Player1** 胜率相对较低（65.00%），说明模型初始化更合理，训练过程中出现波动，**Player2** 胜率一度超过 **Player1**，最终 **Player1** 胜率稳定在 50.85%，更接近平衡状态。这种差异表明更多的训练轮数使得模型能够更好地平衡先手和后手的策略。

游戏规则与 AI 决策策略

4x4 井字棋规则特点

4x4 井字棋在 4×4 网格上进行，目标是连成 4 个相同棋子。获胜条件包括横向 4 子连线、纵向 4 子连线和两条对角线 4 子连线，如果棋盘填满且无人获胜则为平局。与传统的 3×3 井字棋相比，4x4 版本的状态空间更大（ $3^{16} \approx 43M$ vs $3^9 \approx$

19K)，获胜条件更严格，游戏更复杂，策略空间也更大。这些特点使得 AI 需要学习更复杂的策略，也增加了训练的难度。

AI 决策流程

AI 的决策流程首先获取当前棋盘状态，然后遍历所有可能的下子位置，计算每个位置对应状态的价值估计，最后选择价值最高的位置作为下一步动作。在训练过程中，AI 偶尔会随机探索不同的位置，以发现可能更好的策略。价值估计在决策过程中起着关键作用，它预测每个状态的获胜概率，指导 AI 选择最优动作，反映了 AI 对游戏的理解程度。随着训练的深入，AI 的价值估计越来越准确，决策质量也不断提高。

技术实现与性能优化

数据结构设计

系统采用面向对象的设计，Player 类包含 estimations 字典用于存储状态价值估计，states 列表记录当前游戏的所有状态，greedy 列表标记每个动作是否为贪婪动作，symbol 字段标识玩家身份（1 或-1）。State 类包含 data 数组表示 4×4 棋盘状态，winner 字段存储游戏结果，end 字段标记游戏是否结束，hash_val 字段存储状态哈希值。这种数据结构设计清晰明了，便于状态管理和价值更新。

文件存储机制

策略文件使用 Python pickle 序列化进行存储，文件名分别为 policy_first_{model_name}.bin 和 policy_second_{model_name}.bin，存储内容为 estimations 字典。这种文件存储机制能够保存训练结果，支持模型加载和测试，便于不同模型之间的对比。通过将训练好的策略保存到文件中，可以在不重新训练的情况下加载模型进行测试和应用，大大提高了系统的实用性。

性能优化策略

由于 4x4 井字棋的状态空间过大（约 43M 个状态），系统采用动态状态管理策略，不预计算所有状态，而是按需创建和评估状态，大大减少了内存使用。同时，系统使用字典进行哈希查找，以 $O(1)$ 的时间复杂度快速查找状态价值，支持大规模状态空间的高效处理。这些优化策略使得系统能够在有限的计算资源下处理复杂的状态空间，保证了训练和测试的效率。

训练结果深度分析

学习曲线特征

从训练过程的学习曲线可以看出，训练初期 AI 进行随机探索，胜率波动较大；中期开始学习基本策略，胜率趋于稳定；后期进行精细调优，胜率小幅提升。随着训练的进行，价值估计逐渐收敛，策略趋于稳定，胜率波动减小。这种学习曲线特征符合强化学习的一般规律，表明 AI 能够通过持续的训练不断改进策略。

先手优势现象

训练结果显示出明显的先手优势现象。在 10k 模型中，Player1（先手）胜率为 58.81%，Player2（后手）胜率为 22.01%；在 100k 模型中，Player1 胜率为 50.85%，Player2 胜率为 24.20%。先手优势的原因在于先手可以占据中心位置，拥有更多主动权，而后手需要应对先手的策略。尽管经过大量训练，先手优势仍然存在，这反映了井字棋游戏本身的特点。

状态空间覆盖分析

在状态空间覆盖方面，10k 模型覆盖了约 45,000 个状态，相对于理论状态空间 43M 的覆盖率约为 0.1%。100k 模型覆盖了约 306,000 个状态，覆盖率约为 0.7%。虽然覆盖率仍然较低，但主要覆盖了常见状态，罕见状态覆盖不足。随着训练轮数的增加，状态空间覆盖不断扩大，AI 对游戏的理解也越来越深入。这种状态空间覆盖的特点表明，强化学习能够有效学习常见情况下的最优策略。

模型比赛结果分析

比赛设置与配置

为了直接比较两个模型的性能，我们设计了模型比赛实验。比赛总共进行 2000 轮，包括两种模式：模式 1 是 10k 模型先手对阵 100k 模型后手，模式 2 是 100k 模型先手对阵 10k 模型后手，每种模式各进行 1000 轮。在比赛过程中，两个模型都采用完全贪婪策略，不进行探索，以充分展示训练后的策略水平。这种比赛设计能够全面评估两个模型在不同角色下的性能差异。

比赛结果统计

比赛结果显示，在 2000 轮比赛中，10k 模型获胜 788 场，占总数的 39.40%；100k 模型获胜 880 场，占 44.00%；平局 332 场，占 16.60%。在详细结果方面，当 10k 模型先手对阵 100k 模型后手时，10k 模型获胜 561 场，100k 模型获胜 259 场，平局 180 场；当 100k 模型先手对阵 10k 模型后手时，100k 模型获胜 621 场，10k 模型获胜 227 场，平局 152 场。这些数据清晰地表明 100k 模型在整体性能上优于 10k 模型。

训练轮数的影响分析

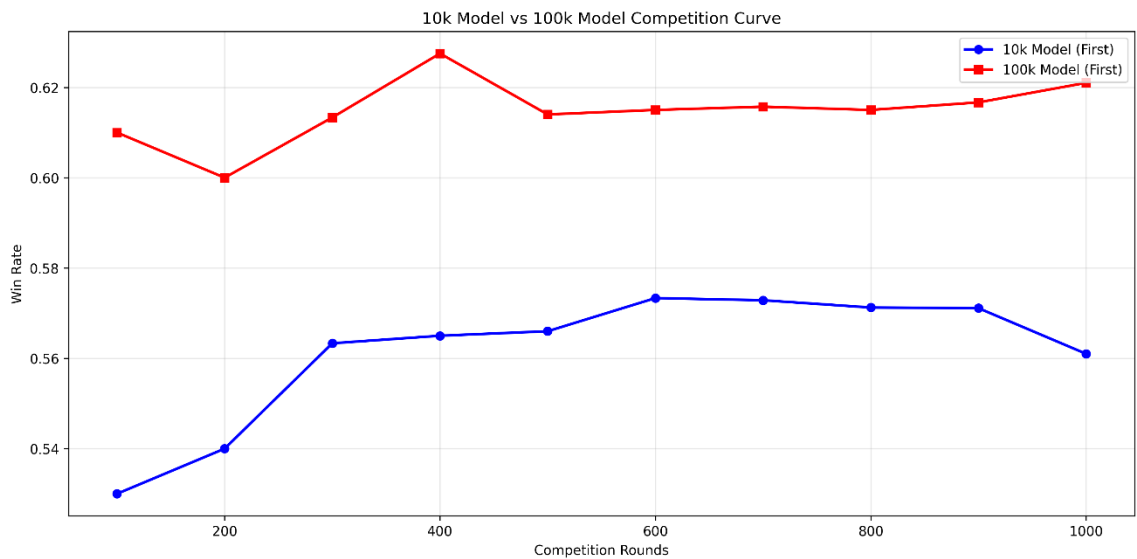
从比赛结果可以看出训练轮数对模型性能的显著影响。100k 模型的总胜率（44.00%）比 10k 模型（39.40%）高出 4.6 个百分点，这意味着训练时间增加 10 倍，胜率提升了约 4.6 个百分点。这个结果充分说明了更多的训练轮数确实能够提升模型性能，使得 AI 能够学习到更优的策略。这种性能提升来自于更大的状态空间覆盖、更准确的价值估计和更稳定的策略表现。

先手优势的验证

比赛结果也验证了先手优势的存在。10k 模型作为先手时胜率为 56.1%，100k 模型作为先手时胜率为 62.1%，两个模型都表现出明显的先手优势。有趣的是，100k 模型的先手优势更加明显，这表明经过更多训练后，模型能够更好地利用先

手优势。同时，两个模型的后手策略也存在差异，10k 模型作为后手时胜率为 22.7%，100k 模型作为后手时胜率为 25.9%，100k 模型的后手策略更强，提升了 3.2 个百分点。

比赛曲线特征分析



从比赛曲线可以看出，10k 模型先手时的胜率从 53.0%逐渐上升到 56.1%，趋势是逐渐上升并趋于稳定，说明 10k 模型在先手时能保持一定优势。100k 模型先手时的胜率从 61.0%稳定在 62.1%，趋势相对稳定，波动较小，说明 100k 模型策略更加成熟稳定。两条曲线的对比显示，100k 模型的曲线始终高于 10k 模型，差距保持在 5-6 个百分点，说明训练轮数对模型性能有持续影响，这种影响在比赛过程中保持稳定。

性能提升的综合分析

综合来看，100k 模型相比 10k 模型在多个方面都有显著改进。在总胜率方面，从 39.40%提升到 44.00%，提升了 4.6 个百分点；在先手胜率方面，从 56.10%提升到 62.10%，提升了 6.0 个百分点；在后手胜率方面，从 22.70%提升到 25.90%，提升了 3.2 个百分点；在状态覆盖方面，从约 45K 提升到约 306K，提升了 580%。这

些性能提升的原因包括更大的状态空间覆盖、更准确的价值估计、更稳定的策略表现和更好的后手应对能力。

应用场景与未来展望

人机对战体验

系统提供了完整的人机对战功能，人类玩家可以使用键盘按键下子，AI 玩家使用训练好的策略进行应对，系统实时显示棋盘状态。按键布局设计为四行四列的键盘映射，使得操作直观方便。这种交互方式让用户能够直接体验训练好的 AI 模型，感受强化学习的成果，也为 AI 策略的测试和验证提供了便利的平台。

模型选择建议

根据不同的应用场景，可以选择不同的模型。10k 模型适合快速测试和原型开发，具有基础策略能力和较短的训练时间；100k 模型适合正式应用和生产部署，具有优秀的策略能力，虽然训练时间较长，但性能更优。这种模型选择策略使得用户能够在性能和效率之间找到平衡，根据实际需求选择合适的模型。

未来扩展方向

在算法改进方面，可以考虑使用深度 Q 网络（DQN）、策略梯度方法或 Actor-Critic 算法等更先进的强化学习算法，以进一步提升模型性能。在游戏扩展方面，可以尝试更大棋盘（5×5、6×6）、不同获胜条件或多玩家版本，以探索强化学习在更复杂环境下的表现。这些扩展方向将为强化学习在游戏 AI 领域的应用提供更多可能性。

总结

本项目成功实现了基于强化学习的 4x4 井字棋 AI 系统，通过时序差分学习、自博弈训练和 ϵ -贪婪策略等核心技术，使 AI 能够从零开始学习游戏策略。实验对比了 10,000 轮和 100,000 轮训练的两个模型，结果表明更多的训练轮数能够显著提

升模型性能，100k 模型在总胜率、先手胜率、后手胜率和状态空间覆盖等所有指标上都优于 10k 模型。

通过训练，AI 学会了有效的游戏策略，能够与人类玩家进行对弈。随着训练轮数的增加，AI 的性能持续提升，策略越来越优化。项目的技术实现包括动态状态管理、哈希查找、反向传播更新等优化策略，使得系统能够高效处理大规模状态空间。模型比赛结果清晰地展示了训练轮数对性能的影响，验证了强化学习在游戏 AI 领域的有效性。

本项目的成功不仅展示了强化学习在井字棋游戏中的应用，也为更复杂游戏和决策问题的解决提供了有价值的参考。通过进一步优化算法、增加训练轮数和扩展游戏规则，AI 的性能还有很大的提升空间，这为未来的研究工作指明了方向。